

WEEK 2: 2D DYNAMIC PROGRAMMING

Advanced Algorithms

ROADMAP

1. Longest Common Substring — a 2D table warm-up
2. Longest Common Subsequence
3. Reducing LCS to LIS
4. Fine-Grained Complexity — why the textbook algorithm may be optimal

LONGEST COMMON SUBSTRING

LONGEST COMMON SUBSTRING

In the longest common substring (LCStr) problem we are given two strings, say “pineapple” and “grappled”.

The goal is to find the length of a longest string that occurs as a *substring* in both input strings.

In this case the answer would be 5. The longest common substring is “apple”.

SUBPROBLEMS

Our first task in the DP strategy is to define useful subproblems.

In previous examples, subproblems were based on *prefixes* of the input.

Now we have two inputs, call them a and b . Let's consider subproblems based on a prefix of a *and* a prefix of b .

2D DP TABLE

	p	i	n	e	a	p	p	l	e
g	→								
r	→								
a	→								
p	→								
p	→								
l	→								
e	→								
d	→								

Indices start at 0. Write $a[0..i]$ for the prefix $a[0], \dots, a[i]$, so $a[0..m-1]$ is all of a .

Define a subproblem on the prefixes $a[0..i]$ and $b[0..j]$, and let $\text{dp}[i][j]$ be its answer.

Intuitively, we will use $\text{dp}[k][\ell]$ for $k < i, \ell < j$ to solve $\text{dp}[i][j]$.

We can solve the subproblems row-by-row from top to bottom and respect the prerequisites.

FIRST ATTEMPT

	p	i	n	e	a	p	p	l	e
g									
r									
a									
p					1				
p									
l									
e									
d									

First attempt: what is the longest common substring on these prefixes?

In this example, the longest common substring on the highlighted prefixes is ‘a’.

CAREFUL Knowing this does not seem helpful for the next subproblem, as this common substring cannot be continued in “grappled”.

SECOND ATTEMPT

	p	i	n	e	a	p	p	l	e
g									
r									
a									
p					0				
p									
l									
e									
d									

Second attempt: what is the longest common substring on these prefixes that *ends* at i in a and at j in b ?

On the highlighted prefixes the answer is now 0: no common substring ends with 'a' in one string and 'p' in the other.

With this definition we can relate $\text{dp}[i - 1][j - 1]$ to $\text{dp}[i][j]$.

SECOND ATTEMPT

	p	i	n	e	a	p	p	l	e
g									
r									
a					1				
p						2			
p							3		
l									
e									
d									

When the last characters match, a common substring ending at $(i - 1, j - 1)$ extends by one letter.

Here 'a', then 'ap', then 'app' build up along the diagonal: 1, 2, 3.

THE LCSTR RECURRENCE

	p	i	n	e	a	p	p	l	e
g	0	0	0	0	0	0	0	0	0
r	0	0	0	0	0	0	0	0	0
a	0	0	0	0	1	0	0	0	0
p	1	0	0	0	0	2	1	0	0
p	1	0	0	0	0	1	3		
l									
e									
d									

Let $dp[i][j]$ be the length of a longest substring ending at index i in a and index j in b .

THE LCSTR RECURRENCE

RECURRENCE

$$\text{dp}[i][j] = \begin{cases} 0 & \text{if } i < 0 \text{ or } j < 0 \\ 0 & \text{if } a[i] \neq b[j] \\ 1 + \text{dp}[i-1][j-1] & \text{otherwise} \end{cases}$$

The final answer is the *maximum* entry anywhere in the table.

LONGEST COMMON SUBSEQUENCE

LONGEST COMMON SUBSEQUENCE

In the longest common subsequence (LCS) problem we are given two strings, say “spaceheater” and “sheeptractor”.

The goal is to find the length of a longest string that occurs as a *subsequence* in both input strings.

In our example, “spactr” is a subsequence of both inputs, as is “sheatr”.

These realize the maximum length of an LCS, which is 6.

diff COMPUTES AN LCS

```
$ diff -U 100 spaceheater.txt \  
    sheeptractor.txt
```

```
s  
-p  
-a  
-c  
-e  
h  
e  
+e  
+p  
+t  
+r  
a  
+c  
t  
-e  
+o  
r
```

One character per line, so a line diff is a character diff.

– deletes, + inserts, and the untouched lines are a common subsequence — here **sheatr**.

Fewest edits = most lines left untouched.

AN LCS, NOT *THE* LCS

```
$ diff -U 100 sheeptractor.txt \  
    spaceheater.txt
```

```
s  
-h  
-e  
-e  
p  
-t  
-r  
a  
c  
+e  
+h  
+e  
+a  
t  
-o  
+e  
r
```

Swap the arguments and `diff` returns the *other* subsequence: `spactr`.

Both have length 6. A longest common subsequence need not be unique.

Which one you get depends on how ties are broken.

SUBPROBLEMS

We again consider subproblems defined on prefixes of the two inputs.

As in other subsequence problems, a good subproblem definition is the *original problem* on these prefixes.

DEFINITION Let $\text{dp}[i][j]$ be the length of a longest common subsequence of $a[0..i]$, $b[0..j]$.

BASE CASE

Let $\text{dp}[i][j]$ be the length of a longest common subsequence of $a[0..i]$, $b[0..j]$.

BASE CASE If $i < 0$ or $j < 0$ then $\text{dp}[i][j] = 0$.

The longest common subsequence of any string with the empty string is 0.

UPDATE: $a[i] \neq b[j]$

We condition on whether or not $a[i] = b[j]$.

If $a[i] \neq b[j]$ then the last element of an LCS of $a[0..i]$, $b[0..j]$ is either not $a[i]$, or not $b[j]$, or neither.

Thus the LCS of $a[0..i]$, $b[0..j]$ is either an LCS of $a[0..i-1]$, $b[0..j]$ or of $a[0..i]$, $b[0..j-1]$.

UPDATE

$$\text{dp}[i][j] = \max(\text{dp}[i][j-1], \text{dp}[i-1][j])$$

UPDATE: $a[i] = b[j]$

Now suppose $a[i] = b[j]$.

We can extend the LCS of $a[0..i-1]$, $b[0..j-1]$ with $a[i] = b[j]$.

UPDATE

$$\text{dp}[i][j] = \max(1 + \text{dp}[i-1][j-1], \\ \text{dp}[i][j-1], \text{dp}[i-1][j])$$

We can further simplify this case.

SIMPLIFYING THE MATCH CASE

Case $a[i] = b[j]$:

Note that $\text{dp}[i][j - 1]$, $\text{dp}[i - 1][j]$ each exceed $\text{dp}[i - 1][j - 1]$ by at most one, thus

$$1 + \text{dp}[i - 1][j - 1] \geq \max(\text{dp}[i][j - 1], \text{dp}[i - 1][j])$$

UPDATE

$$\text{dp}[i][j] = 1 + \text{dp}[i - 1][j - 1] \quad \text{when } a[i] = b[j]$$

FINAL UPDATE

RECURRENCE

$$\text{dp}[i][j] = \begin{cases} 0 & \text{if } i < 0 \text{ or } j < 0 \\ 1 + \text{dp}[i-1][j-1] & a[i] = b[j] \\ \max(\text{dp}[i][j-1], \text{dp}[i-1][j]) & \text{otherwise} \end{cases}$$

The length of an LCS is $\text{dp}[m-1][n-1]$, the bottom-right cell — unlike LCStr, where we took the maximum over the whole table.

FINAL UPDATE

RECURRENCE

$$\text{dp}[i][j] = \begin{cases} 0 & \text{if } i < 0 \text{ or } j < 0 \\ 1 + \text{dp}[i-1][j-1] & a[i] = b[j] \\ \max(\text{dp}[i][j-1], \text{dp}[i-1][j]) & \text{otherwise} \end{cases}$$

For strings of length m and n this finds the length of an LCS in time $O(mn)$.

We have reasons to think this algorithm is essentially optimal (discussed later).

EXAMPLE

		s	h	e	e	p	t	r	a	c	t	o	r
	0	0	0	0	0	0	0	0	0	0	0	0	0
s	0	1	1	1	1	1	1	1	1	1	1	1	1
p	0	1	1	1	1	2	2	2	2	2	2	2	2
a	0	1	1	1	1	2	2	2	3	3	3	3	3
c	0	1	1	1	1	2	2	2	3	4	4	4	4
e	0	1	1	2	2	2	2	2	3	4	4	4	4
h	0	1	2	2	2	2	2	2	3	4	4	4	4
e	0	1	2	3	3	3	3	3	3	4	4	4	4
a	0	1	2	3	3	3	3	3	4	4	4	4	4
t	0	1	2	3	3	3	4	4	4	4	5	5	5
e	0	1	2	3	4	4	4	4	4	4	5	5	5
r	0	1	2	3	4	4	4	5	5	5	5	5	6

→ along a row is an *insert*; ↓ down a column is a *delete*.

spaceheater
shpaceheater
shepaceheater
sheepaceheater
sheeptaceheater
sheeptraceheater
sheeptraceater
sheeptracater
sheeptractor
sheeptractor

The highlighted path spells out an edit sequence turning “spaceheater” into “sheeptractor”.

TRACEBACK: RECOVERING THE LCS

The table gives the *length* of an LCS. To recover the LCS itself, walk backwards from $\text{dp}[m - 1][n - 1]$:

- If $a[i] = b[j]$, this character is part of the LCS: output it and step diagonally to $(i - 1, j - 1)$.
- Otherwise step to whichever of $(i - 1, j)$ and $(i, j - 1)$ holds the larger value — that is where the max came from.
- Stop when $i < 0$ or $j < 0$; the output, reversed, is an LCS.

TRACEBACK IN CODE

```
while i ≥ 0 and j ≥ 0:
    if a[i] == b[j]:                # match: part of the LCS
        out.push_back(a[i]); --i; --j
    else if dp(i-1, j) ≥ dp(i, j-1):
        --i                        # LCS unchanged without a[i]
    else:
        --j                        # LCS unchanged without b[j]
reverse(out)
```

The walk takes $O(m + n)$ steps.

LIS AND LCS

LIS AND LCS

Do you think LIS and LCS have anything to do with each other?

We can use an algorithm for LIS to solve LCS.

Let's look at the reduction with an example.

REDUCTION

	0	4	7	8	3,2	1	3,2	7	9,5	3,2	11,6	
$x =$	s	p	a	c	e	h	e	a	t	e	r	
	0	1	2	3	4	5	6	7	8	9	10	
$y =$	s	h	e	e	p	t	r	a	c	t	o	r
	0	1	2	3	4	5	6	7	8	9	10	11

Blue: indices. Orange: the positions where each letter of x appears in y , listed in decreasing order.

For every letter in x , write down the positions it appears in y **in decreasing order**.

Form a list of numbers z by replacing each character of x with this list of positions.

REDUCTION

	0	4	7	8	3,2	1	3,2	7	9,5	3,2	11,6
$x =$	s	p	a	c	e	h	e	a	t	e	r
	0	1	2	3	4	5	6	7	8	9	10

$y =$	s	h	e	e	p	t	r	a	c	t	o	r
	0	1	2	3	4	5	6	7	8	9	10	11

$z =$	0	4	7	8	3	2	1	3	2	7	9	5	3	2	11	6
-------	---	---	---	---	---	---	---	---	---	---	---	---	---	---	----	---

Question 1: what is the size of z ?

SIZE OF z

Let r_i be the number of times $x[i]$ appears in y .

Then the size of z is

$$r = \sum_{i=0}^{m-1} r_i$$

SIZE OF INPUT

The size of z can range from 0 (the two strings are disjoint) to mn .

Example 1

a	b	c
---	---	---

d	e	f
---	---	---

$$r = 0$$

Example 2

a	a	a
---	---	---

a	a	a
---	---	---

$$r = 9$$

SIZE OF INPUT

The size of z can range from 0 (the two strings are disjoint) to mn .

In the situation of comparing files with `diff`, we typically don't expect a line of one file to appear many times in the other file.

If each line of the first file appears a constant number of times in the second then the size of z is $O(m)$.

BUILDING z

Question 2: how much time does it take to make z ?

- Put the elements of x into a map with value an empty list — $O(m \log m)$.
- Iterate through y . If $y[i]$ is in the map, add i to the list associated with $y[i]$ — $O(n \log m)$.
- Iterate through x to build z — $O(m \log m + r)$.

CORRECTNESS

	0	4	7	8	3,2	1	3,2	7	9,5	3,2	11,6
$x =$	s	p	a	c	e	h	e	a	t	e	r
	0	1	2	3	4	5	6	7	8	9	10

$y =$	s	h	e	e	p	t	r	a	c	t	o	r
	0	1	2	3	4	5	6	7	8	9	10	11

$z =$	0	4	7	8	3	2	1	3	2	7	9	5	3	2	11	6
-------	---	---	---	---	---	---	---	---	---	---	---	---	---	---	----	---

The highlighted increasing sequence 0, 4, 7, 8, 9, 11 picks out “s p a c t r” — a common subsequence of x and y .

CORRECTNESS

CLAIM Any increasing sequence in z corresponds to a common subsequence of x and y , and vice versa.

The decreasing order within each letter's block matters: an increasing sequence can use at most one number per block, so each letter of x is matched at most once.

A LIS of z is a longest common subsequence of x and y .

SUMMARY

We can find the LCS of x and y by a reduction to LIS in time

$$O(m \log m + n \log m + r \log m)$$

The last term is the LIS on z : patience sorting keeps one pile per unit of LIS length, and that length is at most m , so each of the r insertions costs $O(\log m)$.

In applications where r is $O(m + n)$ this can be much better than the $\Theta(mn)$ time algorithm we saw previously.

IN PRACTICE: `diff`

TAKEAWAY This is how the original Unix `diff` compared files (Hunt–Szymanski, 1976): lines rarely repeat, so r is small and LCS-via-LIS runs in near-linear time.

Modern tools took a different route — `git diff` defaults to Myers' $O(ND)$ algorithm — though `git diff --patience` still finds an LIS over uniquely-matching lines.

FINE-GRAINED COMPLEXITY

SATISFIABILITY

Probably the most famous problem in theoretical computer science.

In the SAT problem the input is a formula that is an AND of ORs of variables and their negations.

$$(x_3 \vee x_2 \vee x_4) \wedge (\neg x_4 \vee x_1 \vee \neg x_2) \wedge (\neg x_3 \vee \neg x_4 \vee \neg x_1)$$

Is there a Boolean assignment to the variables that makes the formula true?

In this example, the assignment

$x_1 = 1, x_2 = 0, x_3 = 1, x_4 = 0$ works.

3-SAT

$$(x_3 \vee x_2 \vee x_4) \wedge (\neg x_4 \vee x_1 \vee \neg x_2) \wedge (\neg x_3 \vee \neg x_4 \vee \neg x_1)$$

This is an example of a **3-SAT** formula because each OR clause involves 3 variables or their negations.

We don't believe that it is possible to solve every 3-SAT formula on n variables in time polynomial in n .

Whether this is possible is equivalent to the question of whether P is equal to NP.

ALGORITHMS FOR 3-SAT

The trivial algorithm for 3-SAT takes time $\text{poly}(n) 2^n$: try all 2^n possible assignments and see if any of them work.

We can beat brute force: PPSZ solves 3-SAT in randomised time $O(1.308^n) = O(2^{0.388n})$.

Paturi, Pudlák, Saks and Zane, “An Improved Exponential-Time Algorithm for k -SAT”; analysis for general 3-SAT due to Hertli.

After a lot of effort, however, we don’t know of any algorithms faster than exponential time 2^{cn} .

EXPONENTIAL TIME HYPOTHESIS

Impagliazzo and Paturi formalized this barrier in the **exponential time hypothesis**.

ETH There is a constant $c > 0$ such that solving 3-SAT requires time $\Omega(2^{cn})$.

This is a much stronger statement than $P \neq NP$.

Impagliazzo and Paturi, “On the Complexity of k -SAT”.

STRONG EXPONENTIAL TIME HYPOTHESIS

In k -SAT we allow the ORs of k variables or their negations.

The larger k is, the harder the problem becomes.

SETH For every $\varepsilon > 0$ there is a k such that solving k -SAT requires time at least $2^{(1-\varepsilon)n}$.

BACK TO LCS

THEOREM If there exists $\varepsilon > 0$ such that for any two strings x, y of size n we can compute the length of an LCS of x and y in time $O(n^{2-\varepsilon})$, then SETH is false.

A relatively small speedup to the basic DP solution for LCS would also imply a breakthrough in algorithms for SAT.

Abboud, Backurs and Vassilevska Williams, “Tight Hardness Results for LCS and Other Sequence Similarity Measures”; Bringmann and Künnemann, “Quadratic Conditional Lower Bounds for String Problems and Dynamic Time Warping”.

A WEB OF SETH-HARD PROBLEMS

There is now a long list of problems for which the best known algorithm is optimal if SETH holds.

PROBLEM	BEST KNOWN	SETH BARRIER
LCS	$O(n^2)$	no $O(n^{2-\varepsilon})$
Edit distance	$O(n^2)$	no $O(n^{2-\varepsilon})$
Fréchet distance	$O(n^2)$	no $O(n^{2-\varepsilon})$
Graph diameter	$\tilde{O}(mn)$	no subquadratic $3/2$ -approx.

Vassilevska Williams, “On some fine-grained questions in algorithms and complexity”.

CONNECTIONS

SAT and LCS are both extremely well studied problems of practical interest.

Why is SETH a more compelling hypothesis than that the DP algorithm for LCS is optimal?

The real value here is the **connection** between problems: we are stuck in finding better algorithms for LCS for the **same reason** as in SAT.